

■ GenAI 4+ Years Interview — Set 9

431. Your RAG answer is correct, but the source citation is wrong. How do you fix it?

Answer:

"I would trace the citation metadata from ingestion through retrieval to generation. Each chunk should retain document ID, page number, section, and version. I would make the application generate citations from retrieved metadata rather than asking the LLM to invent source references."

432. The LLM cites a document that wasn't retrieved. What do you do?

Answer:

"I would prevent the model from generating arbitrary citations. The application should provide only valid source IDs and map them to retrieved documents after generation."

433. How do you ensure citations are accurate?

Answer:

"I would attach source metadata to every retrieved chunk and validate that every citation in the response corresponds to an actual retrieved source."

■ Retrieval + Query Understanding

434. User asks a vague question: "What about the policy?"

Answer:

"I would use conversation history to understand the reference if available. If the question is still ambiguous, I would ask a clarification question rather than retrieve unrelated documents."

435. User asks: "What is the limit?"

Answer:

"I would use previous conversation context to identify what 'limit' refers to. If multiple possible limits exist, I would ask the user to clarify."

436. Should conversation history always be sent to the LLM?

Answer:

"No. Sending the entire history increases token usage and latency. I would retain only relevant conversation context or summarize older messages."

437. Conversation history becomes 100 messages long. What do you do?

Answer:

"I would use conversation summarization, sliding windows, or selective retrieval of relevant previous messages instead of sending the entire conversation."

■ Memory

438. What is the difference between RAG and conversation memory?

Answer:

"RAG retrieves external knowledge such as company documents. Conversation memory stores information from the current or previous interaction. They solve different problems."

439. Should user conversation history go into the vector database?

Answer:

"It can be useful for semantic memory, but I wouldn't automatically store everything. I would decide what information needs long-term retrieval and apply privacy, retention, and authorization controls."

440. User says "Remember my preferred report format." How could you handle it?

Answer:

"I could store that preference in a structured user-preference store rather than treating every conversation message as knowledge."

■ Query Rewriting

441. What is query rewriting?

Answer:

"Query rewriting transforms the user's original question into a clearer search query that is better suited for retrieval."

442. When is query rewriting useful?

Answer:

"It is useful for ambiguous, conversational, or incomplete queries, especially when the current question depends on previous conversation context."

443. Could query rewriting introduce errors?

Answer:

"Yes. The rewritten query might change the user's intent. I would evaluate rewritten queries and preserve the original question for final answer generation."

■ Multi-Query Retrieval

444. What is multi-query retrieval?

Answer:

"Instead of generating one search query, the system creates multiple variations of the question and retrieves documents for each variation. The results are combined and ranked."

445. Why use multi-query retrieval?

Answer:

"It can improve recall when the user's wording doesn't closely match the terminology used in the documents."

446. What is the disadvantage?

Answer:

"It increases retrieval operations, latency, and potentially cost. I would use it selectively for queries where recall is more important."

■ Parent-Child Retrieval

447. What is parent-child chunking?

Answer:

"A document can be divided into smaller child chunks for accurate retrieval while keeping a larger parent section for context. When a child matches, the system can return the parent content."

448. Why is parent-child retrieval useful?

Answer:

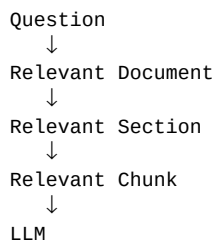
"Small chunks improve retrieval precision, while larger parent sections preserve enough context for answer generation."

■ Hierarchical RAG

449. What is hierarchical retrieval?

Answer:

"Instead of searching all chunks directly, the system can first identify relevant documents or sections and then search within those sections for specific chunks."



450. When would you use hierarchical retrieval?

Answer:

"For very large document collections or documents with strong hierarchical structure such as manuals, policies, and technical documentation."

■ Hybrid Search

451. Why does semantic search sometimes fail with product IDs?

Answer:

"Product IDs, ticket numbers, error codes, and exact names are often better handled by lexical or keyword search. That's why hybrid search can be useful."

452. Give an example where keyword search is better than vector search.

Answer:

"If the user asks for error code `ERR-5007`, exact keyword matching may be more reliable than semantic similarity."

453. Give an example where vector search is better.

Answer:

"If the user asks 'How can I reset my password?', semantic search can retrieve documents discussing password recovery even if they use different wording."

■ Production Debugging

454. The API returns 200 but the answer is empty. What do you check?

Answer:

"I would check the complete pipeline: request parsing, retrieval results, context construction, LLM response, response parsing, and output serialization. A 200 status doesn't guarantee a successful business response."

455. Vector search returns zero results. What do you check?

Answer:

"I would check the query embedding, vector index availability, similarity threshold, metadata filters, embedding dimension, and whether the expected documents were successfully indexed."

456. Vector search returns results but similarity scores are extremely low.

Answer:

"I would check whether the query and documents use the same embedding model, whether preprocessing is correct, whether the chunks contain useful text, and whether the similarity metric is configured correctly."

457. Why is using a different embedding model for queries and documents a problem?

Answer:

"The vectors may belong to different embedding spaces and may not be meaningfully comparable. The same compatible embedding model should normally be used for both document and query embeddings."

■ Indexing

458. How do you know whether a document was successfully indexed?

Answer:

"I would track ingestion status and verify document ID, chunk count, embedding status, and vector database records. I would also run a retrieval test using known questions from that document."

459. Ingestion says successful, but the document cannot be retrieved. What do you check?

Answer:

"I would verify that embeddings were actually generated, vectors were inserted, metadata filters are correct, the index was committed or refreshed if required, and the query uses the same embedding configuration."

460. How do you handle partial ingestion failures?

Answer:

"I would track processing status at the document or chunk level. Failed items can be retried independently rather than reprocessing the entire dataset."

■ Database Consistency

461. Metadata says a document is active, but the vector database doesn't contain its chunks.

Answer:

"This is a consistency issue. I would reconcile the metadata store and vector index and use ingestion status tracking so a document isn't marked active until indexing succeeds."

462. How would you avoid serving a document before indexing completes?

Answer:

"I would maintain document states such as `UPLOADED`, `PROCESSING`, `INDEXED`, and `FAILED`. Retrieval should only consider documents in the `INDEXED` state."

■ Distributed Systems

463. Two workers process the same document simultaneously. What problem can occur?

Answer:

"It can create duplicate embeddings and duplicate vector records. I would use idempotency keys, distributed locks, or unique constraints to prevent concurrent duplicate processing."

464. What is distributed locking?

Answer:

"A distributed lock ensures that only one worker can process a particular resource at a time across multiple application instances."

465. When would you use a lock in RAG ingestion?

Answer:

"For operations such as rebuilding a document index or processing the same document version where concurrent workers could create inconsistent or duplicate state."

■ Queue Scenarios

466. Your ingestion queue has 1 million pending jobs. What do you check?

Answer:

"I would check worker capacity, processing rate, job failures, downstream API limits, queue age, and whether autoscaling is working. I would increase workers carefully while respecting downstream rate limits."

467. Queue depth keeps increasing even after adding workers. Why?

Answer:

"The bottleneck may be downstream, such as the embedding API, vector database, network, or document extraction. Adding more workers won't help if another component is limiting throughput."

468. What metric would you monitor for a queue?

Answer:

"Queue depth, oldest message age, processing rate, failure rate, retry count, and dead-letter queue size."

■ Rate Limits

469. Embedding API returns HTTP 429. What do you do?

Answer:

"I would respect the provider's rate limits, reduce concurrency, use exponential backoff, batch requests where supported, and monitor request rates."

470. Why shouldn't you immediately retry a 429 request 10 times?

Answer:

"It can make the rate-limit situation worse and increase latency. I would use controlled retries with backoff and honor any retry-after information provided by the service."

■ Streaming

471. Why use streaming for LLM responses?

Answer:

"Streaming allows users to see the response as it is generated rather than waiting for the complete response. It improves perceived latency."

472. Does streaming reduce the actual model generation time?

Answer:

"Not necessarily. It mainly improves time-to-first-token and user experience. The total generation time may remain similar."

473. What if streaming disconnects halfway through?

Answer:

"I would handle connection interruptions gracefully, log the partial response status, and determine whether the request can be resumed or safely retried."

■ PII & Sensitive Data

474. Your PDF contains customer phone numbers and email addresses. What do you do?

Answer:

"I would identify whether the data is necessary for retrieval. If not required, I would sanitize or redact it before embedding. I would also apply access controls and secure logging."

475. Why can sensitive data in embeddings be a concern?

Answer:

"Embeddings are derived from the underlying content, so sensitive information may still be represented in the vector or associated metadata. Access control and data governance must therefore apply to the vector store as well."

476. Should you log the complete user prompt?

Answer:

"Not always. If prompts can contain sensitive information, I would minimize logging, redact sensitive fields, and follow the organization's data-retention policy."

■ PII Leakage

477. LLM accidentally exposes a customer's phone number. What do you investigate?

Answer:

"I would determine whether the number came from retrieved context, conversation history, cached data, or model generation. Then I would fix the relevant access-control or filtering issue and add a regression test."

■ Guardrails

478. What are guardrails?

Answer:

"Guardrails are controls around an AI system that restrict unsafe, unauthorized, invalid, or unwanted behavior."

Examples:

- Input validation
- Authorization
- Prompt injection protection
- PII filtering
- Output validation
- Content filtering
- Rate limiting

479. Are guardrails only applied after the LLM response?

Answer:

"No. They can be applied before retrieval, before the LLM, during tool execution, and after generation."

■ Tool Calling

480. What is function or tool calling?

Answer:

"It allows an LLM to request a predefined application function, such as querying an order API or retrieving database information, using a structured schema."

481. Why is tool calling better than allowing the LLM to execute arbitrary code?

Answer:

"Tools provide controlled, predefined operations with validation and authorization. Arbitrary code execution creates significant security and reliability risks."

482. User asks the agent to delete an order. What should happen?

Answer:

"The application should verify authorization, validate the order, possibly require user confirmation, and call a restricted delete operation. The LLM should not directly perform unrestricted database operations."

■ Agent Loops

483. Your agent keeps calling the same tool repeatedly. What do you do?

Answer:

"I would set maximum tool-call limits, detect repeated calls, improve the tool descriptions and stopping criteria, and log the agent's decision sequence."

484. Why are maximum iteration limits important for agents?

Answer:

"They prevent infinite loops, excessive API calls, high cost, and runaway execution."

■ Agent vs RAG

485. Is RAG an agent?

Answer:

"Not necessarily. RAG is primarily a retrieval and generation architecture. An agent can use RAG as one of its tools while also calling APIs, databases, or other services."

486. When would you use an agent instead of simple RAG?

Answer:

"If the application needs dynamic multi-step actions, tool selection, API calls, database queries, or workflow execution. For straightforward document question-answering, simple RAG is usually easier to control."

■ Final Difficult Scenarios

487. Your RAG system is accurate but users don't trust it.

Answer:

"I would improve transparency by providing citations, document names, page numbers, confidence indicators where appropriate, and clear explanations of when information isn't available."

488. Your system has excellent evaluation scores but production users complain.

Answer:

"I would check whether the evaluation dataset represents real production traffic. I would collect real user queries, identify failure patterns, update the evaluation dataset, and rerun the evaluation."

■ ****Very strong interview point.****

489. Why can offline evaluation be misleading?

Answer:

"The test dataset may not represent real-world queries, document changes, user behavior, or edge cases. Production monitoring and continuous evaluation are therefore necessary."

490. How would you continuously improve a RAG system?

Answer:

"I would collect user feedback, analyze failed queries, monitor retrieval and answer metrics, update the evaluation dataset, improve chunking/retrieval/prompts, and continuously test changes before production."

■ 491. Your manager says: "Just increase Top-K to fix retrieval." What do you say?

Answer:

"Increasing Top-K may improve recall, but it can also introduce irrelevant context, increase token cost, and increase latency. I would first identify why relevant documents are being missed and evaluate chunking, embeddings, query rewriting, filtering, and reranking."

■ 492. Your manager says: "Just use a bigger LLM." What do you say?

Answer:

"A larger model may improve generation, but it won't reliably solve missing or incorrect retrieval. I would first determine whether the problem is retrieval or generation and optimize the failing component."

■ 493. Your manager says: "Send more context to reduce hallucination." What do you say?

Answer:

"More context isn't always better. Irrelevant context can distract the model and increase cost. I would retrieve high-quality evidence, rerank it, and send only the context needed to answer the question."

■ 494. Your manager says: "Set temperature to zero and hallucinations will disappear."

Answer:

"Temperature zero can make generation more deterministic, but it doesn't guarantee factual correctness. If the retrieved context is incorrect or missing, the model can still produce an unsupported answer. Retrieval quality and grounding controls are more important."

■ 495. Your RAG system suddenly starts answering with outdated policies.

Answer:

"I would check document versioning, ingestion status, active/inactive metadata, indexing timestamps, cache entries, and retrieval filters. I would ensure the current document version is indexed and older versions are excluded unless explicitly requested."

■ 496. Current policy is indexed, but old policy is still being retrieved.

Answer:

"I would inspect the metadata filters and ranking logic. If the current policy should always take priority, I would filter inactive versions or boost the current effective version appropriately."

■ 497. How do you handle an emergency document update?

Answer:

"I would prioritize the document in the ingestion queue, process and index it quickly, invalidate affected caches, verify retrieval using known queries, and monitor production responses."

■ 498. Your vector database becomes unavailable during a production incident. What do you do?

Answer:

"I would check service health and connectivity, fail over if a replica or secondary store exists, and prevent unsupported LLM responses. I would communicate degraded service and restore normal retrieval after the database recovers."

■ 499. How would you explain your RAG architecture in 60 seconds?

Answer:

"Documents are uploaded to storage and processed asynchronously. We extract the text, split it into meaningful chunks, generate embeddings, and store those embeddings with metadata in a vector database. When a user asks a question, we authenticate the user, create a query embedding, perform filtered similarity or hybrid search, rerank the results, and send the most relevant context to the LLM. The LLM generates a grounded response with citations. We monitor retrieval quality, latency, errors, token usage, cost, and security."

■ 500. "Design a production-ready GenAI RAG system."

Best interview answer:

"I would design it as separate ingestion and query pipelines."

>

The ingestion pipeline would receive documents through object storage, trigger asynchronous processing through a queue, extract text/OCR, perform semantic chunking, generate embeddings in batches, and store vectors with metadata such as tenant ID, access level, document version, page number, and effective date.

>

The query pipeline would authenticate and authorize the user, optionally rewrite the query, perform metadata-filtered vector or hybrid search, apply similarity thresholds and reranking, construct a minimal context, and call the LLM with a grounded prompt. The response would be validated and returned with citations.

>

For production, I would add load balancing, auto-scaling, caching, rate limiting, timeouts, retries with exponential backoff, circuit breakers, monitoring, distributed tracing, cost tracking, prompt/model versioning, security controls, and automated evaluation.

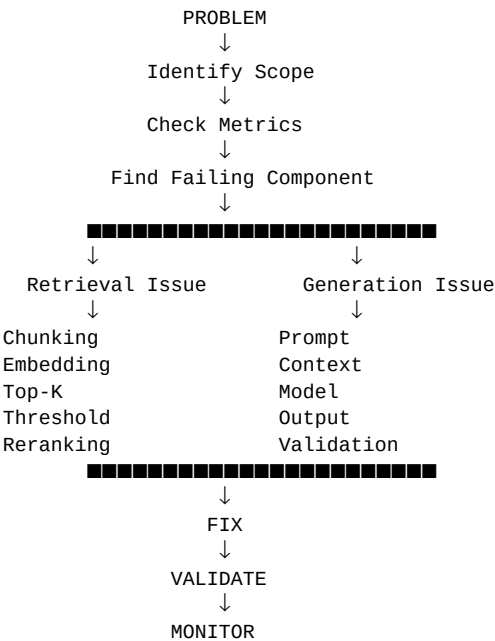
>

For continuous improvement, I would analyze failed queries, user feedback, retrieval metrics, groundedness, correctness, latency, and cost, then make controlled changes and validate them against a regression dataset before deployment."

■ One thing to remember for your interview

When you get a scenario you haven't prepared for, **don't immediately give a solution.**

Use this sequence:



Your strongest interview phrase:

"I would first isolate whether the issue is in ingestion, retrieval, context construction, generation, or infrastructure. I would verify it with metrics and logs before changing the system, then apply a targeted fix and validate the improvement using our evaluation dataset and production monitoring."